

Тема 2. Разветвления и циклы

Разветвления: зачем нужны

Разветвление — это выполнение разного кода в зависимости от условия. Например: есть подписка "pro" — один сценарий, нет подписки — другой.

Инструкция if

Выполняет блок кода только если условие истинно (true).

```
if (condition) {  
  // код выполнится, если condition === true  
}
```

Пример:

```
let price = 0;  
const subscription = "pro";  
  
if (subscription === "pro") {  
  price = 100;  
}  
// price = 100
```

Если условие ложно — блок пропускается, программа идёт дальше.

if...else

Добавляет альтернативный сценарий, если условие ложно.

```
if (grade >= 70) {  
  console.log("Удовлетворительно");  
} else {  
  console.log("Неудовлетворительно");  
}
```

else if

Позволяет проверить несколько условий подряд. Проверка идёт сверху вниз, выполняется код **первого** условия, которое оказалось истинным, остальные не проверяются.

```
if (grade >= 90) {  
  console.log("Отлично");  
} else if (grade >= 80) {  
  console.log("Хорошо");  
} else if (grade >= 70) {  
  console.log("Удовлетворительно");  
} else {  
  console.log("Неудовлетворительно");  
}
```

Логика: «ищу первое подходящее условие, остальное игнорирую».

Тернарный оператор

Короткая замена if...else, когда нужно просто присвоить или вернуть значение.

```
condition ? выражение_если_true : выражение_если_false
```

```
const age = 20;  
const type = age >= 18 ? 'adult' : 'child';
```

Использовать только для простых случаев с ровно двумя вариантами. Для сложной логики — не подходит, ухудшает читаемость.

Оператор switch

Сравнивает выражение с набором значений через строгое равенство (===). Удобен, когда одна переменная имеет много вариантов значений.

```
switch (fruit) {  
  case 'apple':  
    console.log('Apple selected');  
    break;  
  case 'banana':  
    console.log('Banana selected');  
    break;  
  default:  
    console.log('Unknown');  
}
```

Правила:

- break обязателен после каждого case — иначе выполнение "проваливается" (fall-through) в следующий блок;
- default — только один, всегда в конце, break после него не нужен;
- сравнение только на строгое равенство (нельзя >, <).

Fall-through можно использовать намеренно — для группировки нескольких case с одинаковым кодом:

```
switch (day) {  
  case 1:  
  case 2:  
  case 3:  
    console.log('Рабочий день');  
    break;  
  case 6:  
  case 7:  
    console.log('Выходной');  
    break;  
}
```

Когда что использовать

Конструкция	Когда применять
if	универсальный вариант
if...else / else if	несколько условий, разная логика
Тернарный оператор	присвоение/возврат значения, ровно 2 варианта
switch	одна переменная, много вариантов значений, сравнение на равенство

Блочная область видимости (scope)

Переменные, объявленные вне блоков {} — глобальные, доступны везде.

Любая конструкция с `{ }` (if, функции, циклы) создаёт свою локальную область видимости. Переменная, объявленная внутри блока, недоступна снаружи.

Правило доступа: блок видит переменные своей области и всех "родительских" (внешних) областей, но не видит переменные "соседних" или вложенных блоков.

```
const globalVar = "Global";

if (true) {
  const aVar = "A"; // видит globalVar, но не видна снаружи
  if (true) {
    const bVar = "B"; // видит globalVar и aVar
  }
}
// aVar и bVar здесь недоступны
```

Логические операторы

Приведение к логическому типу (Boolean)

Любое значение можно привести к `true` или `false` — явно через `Boolean()`, либо неявно в условиях (`if`, `&&`, `||`).

6 значений, которые всегда дают **false**: `0`, `""`, `NaN`, `null`, `undefined`, `false`. Всё остальное — **true** (в том числе строки `"0"` и `"false"`).

```
Boolean(0);           // false
Boolean("");          // false
Boolean(NaN);         // false
Boolean(null);        // false
Boolean(undefined);   // false

Boolean(3.14);        // true
Boolean("hello");     // true
Boolean("false");     // true (непустая строка!)
```

Оператор && (И)

Проверяет операнды слева направо. Возвращает **первый falsy-операнд**, на котором споткнулся, а если все истинны — **последний (правый) операнд**.

```
"hello" && 5;          // 5   (оба true → вернул правый)
5 && "hello";          // "hello"

"hello" && 0;           // 0   (0 первый falsy)
0 && "hello";          // 0   (0 falsy → правый даже не вычисляется)
```

Практика — проверка нескольких условий одновременно:

```
const a = 20;
a > 10 && a < 30; // true && true -> true
```

Оператор || (ИЛИ)

Проверяет операнды слева направо. Возвращает **первый truthy-операнд**, на котором остановился (правый не вычисляется), а если все ложны — **последний (правый) операнд**.

```
5 || false;    // 5
false || 5;    // 5

0 || false;    // false (оба falsy → вернул правый)
null || "";    // ""
```

Практика:

```
const a = 5;
a < 10 || a > 30; // true || false -> true
```

Оператор ! (НЕ)

Унарный оператор (один операнд справа). Приводит к boolean и инвертирует результат.

```
!true;    // false
!0;       // true
!"";      // true
!"Mango"; // false
```

Практика — проверка «от обратного»:

```
const isBlocked = false;
const canChat = !isBlocked; // true

if (canChat) {
  console.log("Can type in chat!");
}
```

Можно комбинировать с && / ||:

```
const canChat = isOnline && !isBlocked;
```

Кратко: как работают && и ||

Оператор	Если все true	Если есть falsy
&&	вернёт последний (правый) операнд	вернёт первый falsy, на котором споткнулся
	вернёт первый truthy, на котором остановился	вернёт последний (правый) операнд

Методы строк

Свойства vs методы

Свойство — характеристика значения, обращение через точку без скобок: `str.length` (количество символов). Метод — действие над значением, вызывается со скобками: `str.toUpperCase()`. И то, и другое не существует само по себе — только применительно к конкретному значению (строке, массиву и т.д.).

slice()

Возвращает копию части строки, не меняя оригинал.

```
str.slice(startIndex, endIndex)
```

`endIndex` не включается в результат и необязателен (без него — до конца строки). Без аргументов — точная копия строки.

```
const fullName = "Jacob Mercer";
fullName.slice(0, 4); // 'Jaco'
fullName.slice(3, 9); // 'ob Mer'
fullName.slice(1);    // 'acob Mercer' (endIndex не указан)
fullName.slice();     // 'Jacob Mercer' (копия)
```

toLowerCase() / toUpperCase()

Возвращают новый рядок в нижнем/верхнем регистре, оригинал не меняют. Частое применение — нормализация пользовательского ввода перед сравнением строк:

```
const brandName = 'samsung';
const userInput = 'saMsUng';

userInput === brandName; // false
userInput.toLowerCase() === brandName; // true
```

includes()

Проверяет, содержит ли строка подстроку. Возвращает true/false. Чувствителен к регистру.

```
str.includes(substring)

'Jacob Mercer'.includes('Jacob'); // true
'Jacob Mercer'.includes('jacob'); // false (регистр важен)
```

startsWith() / endsWith()

Проверяют, начинается/заканчивается ли строка заданной подстрокой. Тоже чувствительны к регистру. Без аргумента — вернут false.

```
str.startsWith("Hello"); // true
str.endsWith("world!"); // true
```

indexOf()

Возвращает индекс первого вхождения подстроки, либо -1, если не найдена.

```
str.indexOf(substr)

"Welcome to Bahamas!".indexOf("to"); // 8
"Welcome to Bahamas!".indexOf("hello"); // -1
```

trim()

Удаляет пробелы в начале и конце строки. Оригинал не меняет, возвращает новую строку. Полезно для очистки пользовательского ввода из форм.

```
const input = " JavaScript is awesome! ";
input.trim(); // "JavaScript is awesome!"
```

Циклы

Цикл — конструкция для многократного выполнения одного и того же блока кода. Тело цикла — код, который повторяется. Итерация — одно выполнение тела. Условие выхода — определяет, продолжать цикл или остановиться.

Цикл while

Выполняется, пока условие истинно. Условие проверяется **перед** каждой итерацией — если оно ложно с самого начала, тело может не выполниться ни разу.

```
while (condition) {  
  // тело цикла  
}
```

```
let count = 0;  
  
while (count < 10) {  
  console.log(`Count: ${count}`);  
  count += 1;  
}
```

Используется, когда заранее неизвестно точное количество итераций — цикл идёт, пока выполняется условие.

Важно следить, чтобы условие рано или поздно стало ложным — иначе бесконечный цикл.

Цикл do...while

Отличие от while: тело выполняется **минимум один раз**, даже если условие изначально ложно — проверка идёт **после** первой итерации.

```
do {  
  // код  
} while (condition);
```

```
let count = 0;  
  
do {  
  console.log(`Count: ${count}`);  
  count += 1;  
} while (count < 5);
```

Цикл for

Есть переменная-счётчик (обязательно через let, не const).

```
for (initialization; condition; afterthought) {  
  // тело цикла  
}
```

- инициализация — выполняется один раз перед стартом, задаёт счётчик;
- условие — проверяется перед каждой итерацией;
- пост-выражение — выполняется после каждой итерации, обновляет счётчик;
- тело — выполняется, пока условие истинно.

```
for (let i = 0; i <= 20; i += 5) {  
  console.log(i); // 0, 5, 10, 15, 20  
}
```

Обратный отсчёт — меняем направление счётчика:

```
for (let i = 20; i >= 0; i -= 5) {  
  console.log(i); // 20, 15, 10, 5, 0  
}
```

Инкремент (+) и декремент (--)

Увеличивают/уменьшают переменную на 1 и сразу сохраняют результат. Есть префиксная и постфиксная форма — разница в том, **когда** используется значение.

Форма	Поведение
++x (префикс)	сначала увеличивает, потом возвращает новое значение
x++ (постфикс)	сначала возвращает текущее значение, потом увеличивает

```
let x = 5;
const y = ++x; // x = 6, y = 6

let x2 = 5;
const y2 = x2++; // x2 = 6, y2 = 5 (взял старое значение)
```

То же самое для --x / x--, только в сторону уменьшения. В циклах for обычно используют i++ вместо i += 1 — короче.

Оператор break

Немедленно прерывает цикл, управление переходит к коду после цикла.

```
for (let i = 0; i < 10; i++) {
  console.log(i);
  if (i === 5) {
    break; // остановит цикл
  }
}
// выведет 0..5, дальше цикл прервётся
```

break внутри функции прерывает только цикл, а не саму функцию — код после цикла в теле функции продолжит выполняться. Если нужно прервать и цикл, и саму функцию сразу (с возвратом результата) — используется return.

```
function findNumberFromFive(max, target) {
  for (let i = 5; i <= max; i += 1) {
    if (i === target) {
      return i; // прерывает и цикл, и функцию
    }
  }
  console.log("этот код не выполнится, если return сработал раньше");
}
```